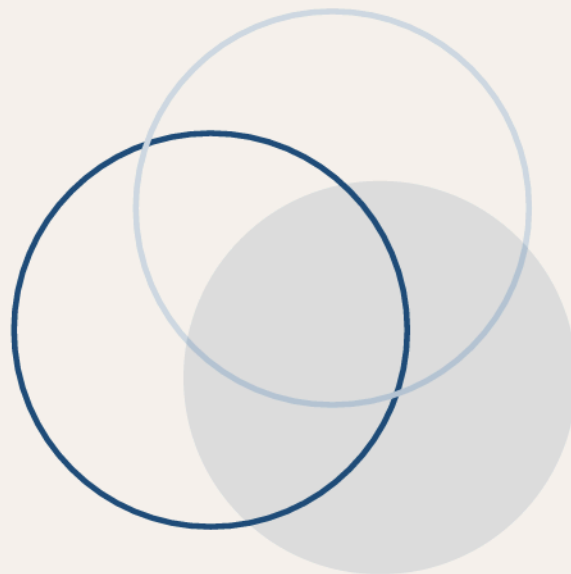




- A. 문제 — 원샷 하니스는 요구 흐름·문맥 상실·회귀 앞에서 사람 손을 요구한다
- B. 설계 — 계획·코딩·테스트 순환, 개발문서와 증거번들이 반복을 잇는다
- C. 증거 — 세 하니스-모델 구성, 세 벤치마크에서 일관된 향상
- D. 시사점 — 전달은 엄격하게, 실행은 유연하게; 기록은 코드만큼 산다

며칠짜리 자율 소프트웨어 개발과 지속 개선 — Harness-of-Harness 정독

하니스의 하니스

차례

들어가며	3
01 제1장 — 원샷의 한계	5
02 제2장 — 세 역할과 하나의 문서	8
03 제3장 — 며칠짜리 개발의 설계 교훈	11
04 제4장 — 시험대	14
05 제5장 — 숫자가 말하는 것	16
06 제6장 — 진행 중인 사례: 퓨즈포인트	21
07 맺음말 — 우리 시스템에 남는 것	24

DAY

들어가며

2026년의 코딩 에이전트는 저장소 하나쯤은 충분히 다룬다. 기존 코드베이스 안에서 이슈를 찾아 고치고, 여러 파일에 걸친 변경을 조율하고, 테스트를 돌려 통과를 확인한다. 그런데 이 능력에는 전제가 깔려 있다. 이미 존재하는 코드베이스라는 것이다. 목표가 분명하고, 제약이 정해져 있고, 성공 판정이 뚜렷한 환경 말이다.

이 책이 다루는 논문 『Harness of Harness』는 그 전제를 건너낸다. 요구사항 문서 한 장을 주고, 빈 작업 공간에서 완전한 소프트웨어를 만들라고 시킨다. 흔히 그린필드(greenfield) 개발이라 부르는, 처음부터 끝까지 스스로 길을 찾아야 하는 과제다. 원샷으로 한 번 실행되는 하니스(harness)는 이 과제에서 오래 버티지 못한다. 요구사항은 흘러가고, 문맥은 잃어버리고, 도구는 환각을 일으키고, 수정이 이전 작업을 무너뜨린다. 그러면 사람이 끼어들어 상태를 살피고, 프롬프트를 고치고, 다시 실행해야 한다.

논문이 제안하는 답은 Harness-of-Harness(HoH)다. 기존 하니스를 새로 만드는 대신, 그 하니스 바깥을 한 겹 더 감싼 바깥 하니스가 사람이 하던 유지보수를 대신 맡는다. 계획하고, 코딩하고, 검증하는 순환을 반복하면서 산출물과 실행 증거를 다음 반복으로 넘긴다. 이 책은 이 프레임워크가 무엇을 어떻게 조율하는지, 숫자로 어디까지 증명했는지, 그리고 우리 시스템 설계에 무엇을 남기는지를 따라간다.

원문은 상하이 인공지능연구소(Shanghai AI Laboratory) 연구진이 2026년 8월에 공개한 예인쇄본이다. 이 책은 원문을 한국어로 요약·재구성한 정독서이며, 그림은 원문 Figure 1-7을 그대로 가져왔다. 원문 제5장의 사례 연구는 아직 진행 중이라 확정되지 않은 수치가 많은데, 이 책도 그 사실을 그대로 드러낸다.

이 책은 같은 시리즈의 『에이전트는 대화로 일하지 않는다』(SKILL.state 정독)와 한 쌍이 된다. 저쪽이 하니스 **안쪽**의 실행 상태를 다룬다면, 이 책은 하니스 **바깥쪽**이 어떻게 며칠짜리 개발을 지탱하는지를 다룬다.

DAY

01

제1장 — 원샷의 한계

하니스란 무엇인가

코딩 에이전트의 성능을 논할 때 모델만 보면 안 된다. 같은 모델이라도 어떤 하니스에 심느냐에 따라 실전 성과가 크게 갈린다. 하니스는 범용 언어 모델을 쓸모 있는 코딩 에이전트로 바꿔 세우는 운영 틀이다. 모델이 도구를 어떻게 부를지, 문맥과 기억을 어떻게 관리할지, 실행을 어떻게 통제할지, 돌아온 피드백을 어떻게 해석할지, 서브에이전트를 어떻게 조율할지를 하니스가 정한다. Codex, OpenCode, Claude Code 같은 요즘 시스템이 공통으로 갖춘 층이다.

하니스라는 말이 주목받은 건 최근 일이지만, 코딩 에이전트 연구 자체는 오래됐다. 다만 초창기는 함수 완성이나 코드 생성 같은 국소적 보조가 전부였다. 그 뒤로 대규모 코드베이스 탐색, 여러 파일에 걸친 저장소 규모 이슈 해결로 무게중심이 옮겨졌다. 그런데도 이 시기 과제들은 모두 기존 코드베이스를 출발점으로 준다. 환경이 정해져 있고, 목적과 제약이 분명하다.

비어 있는 작업 공간의 문제

논문이 겨냥하는 것은 소프트웨어 개발 **from scratch**, 즉 그린필드 개발이다. 요구사항은 높은 수준으로, 그리고 불완전하게 주어진다. 에이전트는 이것을 완전하고 동작하는 소프트웨어 시스템으로 바꿔야 한다. 장기 계획, 조율, 실행, 적응이 소프트웨어 수명주기 전체에 걸쳐 필요하다. 기존 벤치마크가 단일 이슈 해결로 개발을 축소해 온 것과 대비되는, 훨씬 열린 형태의 개발이다.

여기서 질문이 나온다. 기존 코딩 에이전트 하니스를 어떻게 써서 개발 **from scratch**를 지탱할 수 있는가?

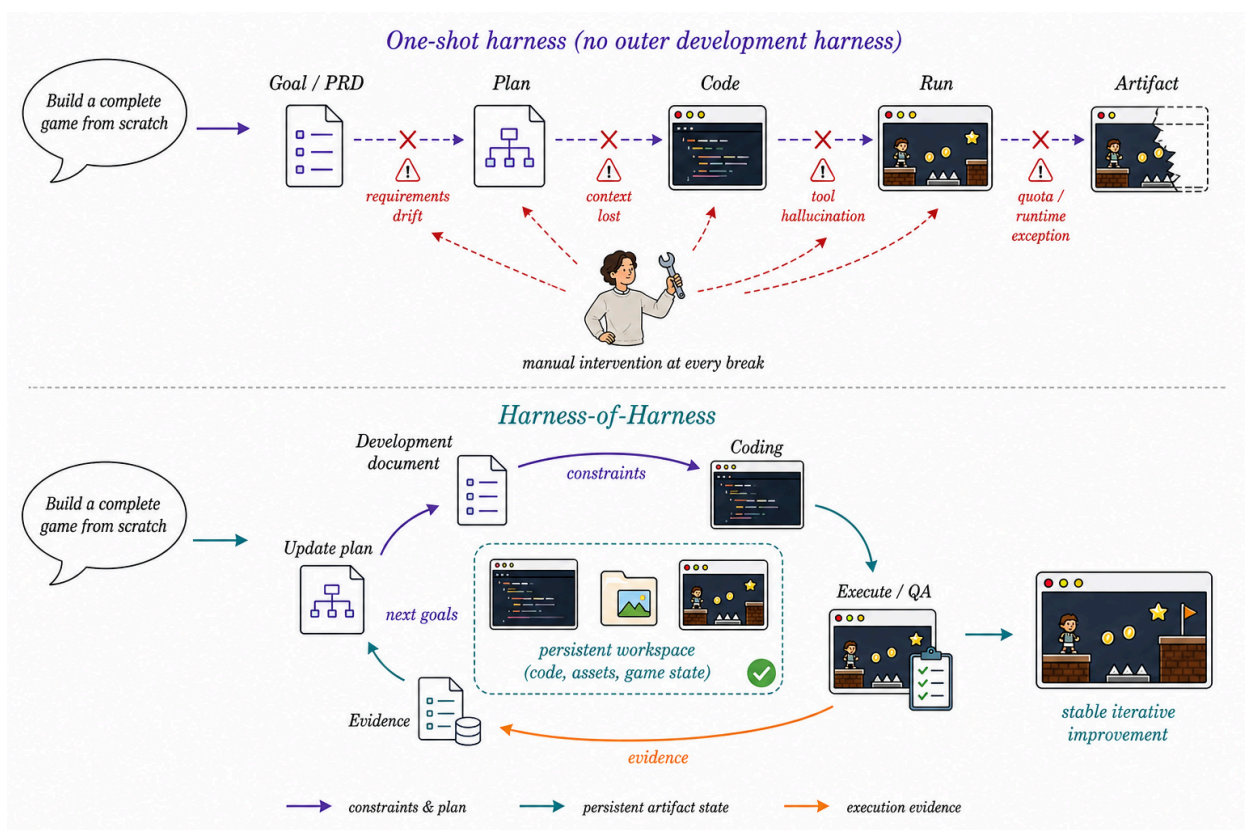


그림 1-1 원샷 하니스와 Harness-of-Harness — 사람이 하던 유지보수를 바깥 하니스가 넘겨받는다

그림 1-10이 이 전환을 그린다. 위쪽이 관행이다. 하니스를 한 번 실행(원샷)하고, 요구 흐름·문맥 상실·도구 환각·런타임 예외가 터지면 사람이 파손 지점을 살피고, 쓸 만한 상태를 보존하고, 다음 프롬프트를 고쳐 다시 실행한다. 개발이 선형으로 흐르되 고장마다 사람이 붙는 구조다. 아래쪽이 HoH다. 바깥 하니스가 그 사람의 자리를 대신한다. 산출물 작업 공간을 보존하고, 실행 증거를 다음 개발 지시로 바꿔 넣고, 계획-코딩-테스트 순환을 스스로 돌린다.

■ 왜 반복인가

실제 소프트웨어 개발은 애초에 증분 과정이다. 개발자는 계획을 고치고, 변경을 구현하고, 결과를 검증하는 일을 수명주기 내내 반복한다. HoH는 이 관찰을 설계 원리로 삼는다. 각 반복을 계획(planning)·코딩(coding)·테스트(testing) 세 단계로 잡는다. 계획은 요구사항과 이전 산출물의 실행 증거를 실행 가능한 개발 계획으로 바꾼다. 코딩은 새 구성요소를 구현·통합하며 기존 산출물을 다듬는다. 테스트는 기능성, 사용성, 미적 품질 등 여러 각도에서 산출물을 평가한다. 그 결과 나온 실행 증거와 갱신된 산출물은 다음 반복으로 넘어간다. 소프트웨어만이 아니라 **개발 결정 자체**가 반복마다 개선되는 자기 개선 고리다.

핵심을 좀더 말하면 이렇다. HoH는 새 하니스를 하나 더 설계한 게 아니다. 이미 검증된 하니스(Codex, OpenCode, Pi 같은)를 그대로 두고, 그것이 참여하는 방식을 구조화한다. 하니스가 모델을 구조화하듯, HoH는 하니스의 참여를 구조화한다.

DAY

02

제2장 — 세 역할과 하나의 문서

문제를 식으로 잡기

논문의 정식화는 담백하다. 하니스-모델 구성을 H 라 하자. 하니스 구현, 기반 모델, 역할별 지침, 사용 가능한 도구와 런타임 정책이 모두 여기에 들어가며, 실행 내내 고정된다. 공개 과제 명세를 S (제품 요구사항 문서 PRD와 공개 보조자료), 초기 소프트웨어 산출물을 A_0 (빈 작업 공간일 수도, 기존 저장소일 수도), 반복 예산을 T 라 하면, HoH의 목표는 S 를 만족하는 최종 산출물 A_T 를 만드는 것이다.

$$\text{HoH}[H] : (S, A_0, T) \mapsto A_T$$

요점은 HoH가 H 자체를 바꾸지 않는다는 데 있다. 바뀌는 것은 각 반복에 H 에 공급되는 산출물과 반복별 문맥이다. 기존 하니스에 대한 투자를 그대로 살리는 설계다.

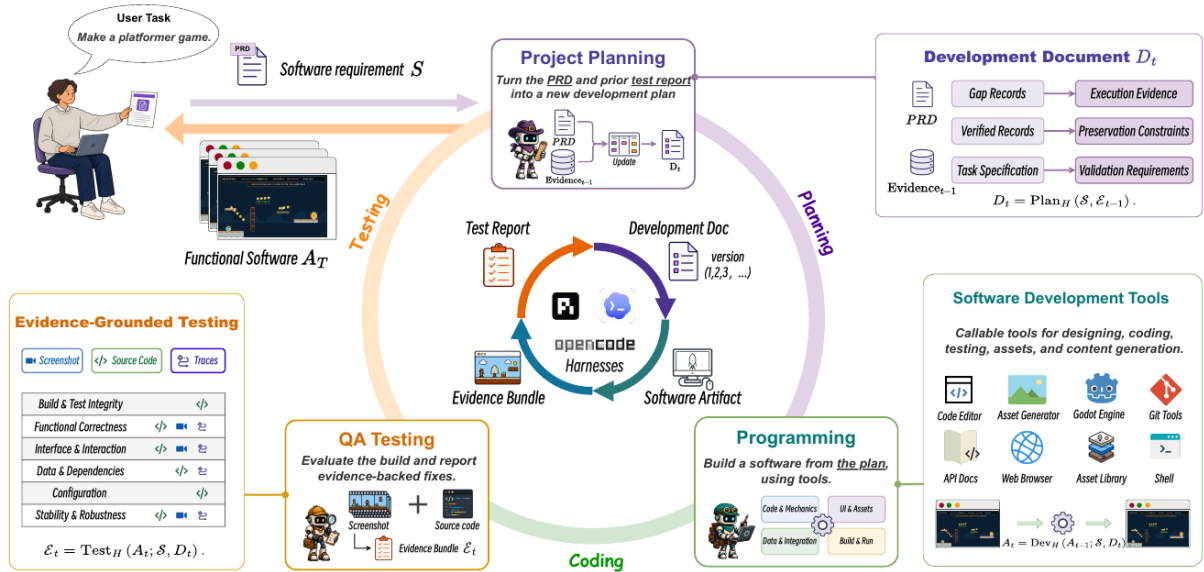


Figure 2 | Overview of Harness-of-Harness (HoH). In each iteration, the *Project Planner* produces an iteration-specific development document, the *Developer* updates the software artifact in the shared project workspace, and the *QA Tester* collects execution evidence. The updated artifact remains in the workspace, while the evidence informs planning in the next iteration.

그림 2-1 HoH 프레임워크 — 프로젝트 계획자, 개발자, QA 테스터가 개발문서와 증거번들을 주고받으며 순환한다

세 역할, 권한은 계층적으로

한 반복 안에서 세 역할이 등장하며, 각 역할은 H 를 한 번 호출해 인스턴스화된다. 셋이 같은 진화하는 프로젝트 작업 공간을 두고 일하되 권한이 다르다.

- **프로젝트 계획자(Project Planner)** — 산출물을 수정하지 않고 계획만 낸다.
- **개발자(Developer)** — 작업 공간을 수정할 수 있다.
- **QA 테스터(QA Tester)** — 산출물을 살피고 실행할 수 있지만 읽기 전용이다.

역할을 나눈 측은 권한이다. 구현과 수용(acceptance)이 분리되어 있어, 개발자의 자가 점검이 완료 판정의 유일한 근거가 되는 일을 막는다.

반복 t 에서 계획자는 **개발문서** D_t 를 만든다. 개발자는 D_t 를 받아 이전 반복에서 기록된 산출물 A_{π_t} (보통 직전 반복의 산출물이며, 직후 산출물이 퇴행 판정을 받으면 더 이른 기록 산출물로 물러난다)에서 개발을 이어가 A_t 를 낸다. QA 테스터는 A_t 를 D_t 의 검증 요구사항에 맞춰 실행·점검하고 증거번들 E_t 를 낸다. D_t 에는 현재 개발 목표, 보존해야 할 기존 기능, 미해결 이슈, 검증 요구사항이 적힌다.

증거번들 — 주장과 근거의 쌍

QA 결과를 그냥 자유 서술 보고서로 두지 않는다. 공개 명세와 개발문서에서 검사 가능한 주장(claim) 집합 C_t 를 뽑고, 증거번들을 주장-증거 레코드의 집합으로 표현한다.

$$E_t = \{(c_i, r_i, s_i) \mid c_i \in C_t\}, \quad s_i \in \{\text{verified}, \text{gap}\}$$

c_i 는 검사 가능한 주장, r_i 는 그 주장에 대해 개발 중 관측 가능한 실행 기록(빌드-테스트 결과, 런타임 트레이스, 스크린샷, 리플레이, 자산 점검), s_i 는 QA 판정이다. 인용한 실행 기록이 주장을 충분히 뒷받침할 때만 verified가 되고, 관측된 실패·미충족 요구·회귀 위험·증거 부족은 gap이 된다. 이 레코드 구조 덕분에 QA 발견마다 관측 근거가 붙어 다니고, 다음 반복 계획이 “무엇을 고치고 무엇을 보존할지”를 근거와 함께 결정할 수 있다.

닫힌 고리 — 두 개의 상태 채널

반복 사이에서 두 가지 상태가 흘러간다. 진화하는 산출물 A_t 와 증거번들 E_t 다. S 와 H 는 실행 내내 고정이고 $E_0 = \emptyset$ 에서 출발하며, 한 반복은 다음처럼 요약된다.

$$(A_{\pi_t}, E_{t-1}) \mapsto (D_t, A_t, E_t), \quad t = 1, \dots, T$$

계획자는 $D_t = \text{Plan}_H(S, E_{t-1})$ 로 문서를 만든다. gap 레코드는 수정 대상과 후속 검증 요구를, verified 레코드는 보존 제약을 정한다. 증거가 다음 문서의 우선순위를 바꿀 수는 있어도 S 자체를 대체하지는 못한다. 루프는 관측 결과에 적응하되 원래 요구에 정렬된 상태를 유지한다.

개발자는 $A_t = \text{Dev}_H(A_{\pi_t}; S, D_t)$ 로 개발한다. D_t 의 정렬된 수정 대상 목록은 빌드 방해 요인과 증거로 확인된 결함을 미충족 요구와 기능 확장보다 앞세우고, 높은 순위가 처리된 뒤 남은 예산을 확장에 쓴다. 보존 제약은 이 모든 변경에 걸쳐 계속 유효하다. 문서의 모든 목표가 H 호출 한 번에 주어지며, 도구 사용과 국소 구현 결정은 하니스 손에 남는다.

QA 테스터는 $E_t = \text{Test}_H(A_t; S, D_t)$ 로 검증한다. 주장별로 $\text{Observe}(A_t, c_i)$ 가 실행 기록을 모으고 $\text{Assess}(c_i, r_i)$ 가 판정을 내린다. 여기서 중요한 제한이 하나 있다. 증거번들은 벤치마크 점수, 은닉 테스트, 평가자 전용 피드백을 **제외**한다. 개발 루프로 평가 신호가 새면 과적합이 되기 때문이다. 중간·최종 산출물 평가는 실행 전체가 끝난 뒤에만 이뤄진다.

전체 절차를 수도코드로 옮기면 다음과 같다.

```
Input : 공개 명세 S, 초기 산출물 A0
Param : 하니스-모델 구성 H (고정), 반복 예산 T
Output: 최종 산출물 AT

E0 ← ∅
for t = 1 .. T:
    # HoH 반복
    Dt ← Plan_H(S, E_{t-1})      # 계획: gap→수정, verified→보존
    At ← Dev_H(A_{π_t}; S, Dt)   # 코딩: 작업 공간에서 이어서
    Ct ← Claims(S, Dt)
    Et ← ∅
    for c_i ∈ Ct:
        # 주장별 검증
        r_i ← Observe(At, c_i)
        s_i ← Assess(c_i, r_i)
        Et ← Et ∪ {(c_i, r_i, s_i)}
return AT
```

반복마다 gap 레코드가 다음 반복의 개발 목표로, verified 기능이 보존 제약으로 이어진다. 산출물 계보와 실행 증거라는 두 채널이 세 역할을 하나의 고리로 묶는다.

DAY

03

제3장 — 며칠짜리 개발의 설계 교훈

소프트웨어는 에이전트 에피소드보다 오래 산다. 높은 수준의 제품 명세에서 그린필드 개발을 하려면 계획·구현·검증·복구가 반복돼야 쓸 만한 산출물이 나오고, 나온 뒤에도 요구와 환경과 사용자의 변화를 따라 수년간 진화한다. 반면 SWE-bench 류 벤치마크는 개발을 “기존 저장소에서 이슈 하나를 해결하면 끝나는 유한 에피소드”로 축소해 왔다. 논문 연구진은 며칠짜리 HoH 실행에서 진행 정체의 원인들을 목격했다. 관련 문맥의 상실, 요구 흐름, 도구와 런타임 실패, 성급한 이슈 종결, 이후 작업을 무효화하는 회귀. 이 장은 그 관찰과 HoH 설계에서 뽑아낸 다섯 가지 실용 교훈을 정리한다.

교훈 1 — 영속 프로젝트 기록은 코드만큼 본질적이다

소스 코드는 시스템이 **지금 무엇인지**는 적지만, **앞으로 무엇이 되어야 하는지**를 판단하는 의도·결정·증거 전체를 담지 않는다. 문맥이 유한하고 프로젝트 기억이 불완전한 에이전트에게 최신 코드베이스만 남겨두면, 호출 때마다 저장소에서 프로젝트 상태를 재구성해야 한다. 그 재구성은 본질적으로 손실이 있다. 요구가 재해석되고, 구현된 기능이 놓치고, 미해결 결함이 잊히고, 검증 안 된 수정이 완성으로 받아들여진다.

그래서 개발문서, 이슈 이력, 테스트 기록은 그것을 낳은 상호작용과 함께 사라지는 게 아니라 산출물과 함께 진화해야 한다. HoH는 목표, 보존 제약, 검증된 동작, 미해결 gap, 산출물 계보를 반복 사이에 운반하는 것으로 이 원칙을 실행한다. 코드베이스와 그 기록이 **하나의 진화하는 프로젝트 상태**를 이루고, 다음 호출은 구현과 함께 그것을 확장하는 데 필요한 지식을 상속받는다.

교훈 2 — 기존 문제 수리와 신규 기능 개발의 균형

장기 개발은 둘 다 필요한데 어느 하나만으로는 부족하다. 관측된 실패만 좇으면 국소 수리에 갇혀 중요한 기능이 미구현으로 남는다. 기능 성장만 좇으면 미해결 방해 요인과 회귀가 점점 복잡해지는 산출물 아래에 묻힌다. 균형점은 고정이 아니라 산출물의 현재 상태에 맞춰 조정돼야 한다.

HoH는 개발 패스마다 전역 명세와 최신 실행 증거를 맞대어 이 절충을 명시적 계획 결정으로 만든다. 증거로 확인된 실패가 무엇을 먼저 고칠지 정하고, 검증된 동작이 이후 변경이 보존해야 할 것을 정하고, 미충족 요구가 산출물이 어디로 자라야 하는지 정한다. 개발문서는 이 요구들을 반복별 우선순위로 배열한다.

교훈 3 — 필요한 것은 안정적 역할이지 계속 부풀는 팀이 아니다

여러 반복에 걸친 신뢰할 만한 진행은 구현과 수용이 분리된 안정적 책임으로 유지될 때 가능하다. 개발자의 자가 점검은 빠른 반복에 도움되지만 현재 변경 안에만 갇혀 있다. 대상 기능은 통과했는데 더 넓은 요구는 미충족이거나, 그 수정이 관련 기능을 깨뜨렸거나, 중단 워크플로가 동작하지 않을 수 있다. 독립적 수용이 이런 국소 점검의 완료 선언을 막는다.

역할 분해의 기준은 하위작업 개수가 아니라 **독립된 결정 경계**다. 서로 다른 권한과 별도 검증 가능한 판단을 내놓을 때만 새 역할이 정당해진다. 그렇지 않은 역할 추가는 맥락과 책임을 산산조각 낼 뿐 신뢰성을 오히려 떨어뜨린다.

교훈 4 — 하니스는 길에서 벗어나게 하되 발목을 잡지 말아야 한다

장기 개발용 하니스는 실행 방식을 규정하지 않으면서 납품은 통제해야 한다. 호출 사이 경계에서는 무엇이 어떻게 인계되는지, 어떤 결과가 공유 프로젝트 상태에 들어갈 수 있는지, 불완전하거나 불안정한 결과를 어떻게 거부·복구하는지를 정한다. 이 제약이 국소 실패가 반복 사이로 퍼지는 걸 막는다. 호출 **안쪽**에서는 추론, 도구 사용, 디버깅 전략, 구현 경로의 통제권을 하니스와 에이전트에 남긴다. 이런 결정은 경계에서 전부 예상할 수 없는 국소 증거에 의존한다. 이 자율성을 지켜야 기반 시스템이 행동을 적응시키고 능력을 다 쓴다. 납품 제약을 내부 실행까지 끌고 들어가면 해 공간이 좁아지고 시스템은 깨지기 쉬워진다. 요컨대 **무엇을 납품하는지는 엄격하게, 어떻게 만드는지는 유연하게**.

교훈 5 — 컨텍스트는 프로젝트 안에서 영속하고, 지식은 프로젝트 사이를 전이한다

요구사항, 개발문서, 이슈 이력, 테스트 보고서, 실행 로그 같은 프로젝트 기록은 코드와 함께 쌓이는 장수 산출물이다. 뒤이은 에이전트 호출은 이 기록으로 목표, 현재 상태, 과거 결정, 미해결 이슈, 검증된 동작을 재구성한다. 그런데 이력이 에이전트 작업 컨텍스트보다 커지면 전체 기록을 프롬프트에 밀어 넣는 순간 낡고 중복되고 상충하는 정보가 함께 들어온다. 에이전트는 뒤쳐진 프로젝트 모델을 세우고 현재 요구에서 흘러간다. 기록은 산출물과 동기화되고 버전 관리되며, 오직 현재와 관련된 증거만 선택적으로 노출되도록 보존돼야 한다.

한편 서로 다른 프로젝트와 과제 유형에서 쌓인 경험 가운데는 반복되는 검증 절차, 디버깅 휴리스틱, 도구 사용 관례, 도메인 워크플로, 실패 패턴이 있다. 이것들은 프로젝트 고유 세부와 함께 끌고 다니는 게 아니라 스킬이나 정책 같은 **재사용 가능한 지식 단위로** 증류돼야 한다. 에피소드 경험이 누적 역량으로 바뀌고, 이후 에이전트가 다른 프로젝트에서 꺼내 쓴다. 프로젝트 고유 컨텍스트는 산출물에 묶여 반복 사이를 살고, 재사용 가능한 지식은 프로젝트 사이를 축적·전이한다.

#	교훈	대응 설계
1	영속 기록은 코드만큼 본질적	개발문서·이슈 이력·증거번들의 반복 간 운반
2	수리와 확장의 균형은 고정기 아니라 상태의 함수	개발문서의 반복별 우선순위 배열
3	안정적 역할, 독립된 결정 경계	계획자·개발자·QA의 권한 분리
4	납품은 엄격, 실행은 유연	경계 계약 + 호출 내부 자율 보장
5	컨텍스트는 영속, 지식은 전이	버전화 기록의 선택적 노출 + 스킬로의 증류

다섯 교훈은 각기 따로 보이지만 한 줄로 묶인다. 하니스가 쥐어야 하는 것은 실행이 아니라 **상태의 계약**이다. 무엇이 다음 반복으로 살아남아야 하는지를 하니스가 정의하는 순간, 에이전트는 며칠짜리 개발을 견딜 수 있다.

DAY

04

제4장 — 시험대

HoH를 평가하려면 개발 과제가 세 축으로 준비돼야 한다. 게임처럼 산출물의 품질을 다면적으로 봐야 하는 것, 저장소 규모의 장기 엔지니어링, 그리고 기존 코드가 아예 없는 상태에서의 재구성. 논문은 세 벤치마크로 이 축을 채우고, 하니스-모델 구성 세 가지로 일반성을 확인한다.

세 벤치마크

벤치마크	과제 형태	본 평가 규모
GameCraft-Bench	자연어 명세에서 완전히 플레이 가능한 Godot 프로젝트 구축	15게임 패밀리 140과제 중 계층 무작위 표집 45과제
FrontierSWE	장기 저장소 엔지니어링	17과제 중 15과제 (구현 4·성능 9·연구 2)
ProgramBench	컴파일된 실행 파일과 문서만으로 동작이 일치하는 코드베이스 재건	클린룸 재구성

GameCraft-Bench의 15 패밀리는 거시 분석을 위해 Action·Timing·Strategy·Simulation·Adventure 다섯 그룹으로 묶인다. FrontierSWE는 범주별 점수와 함께 **Dominance**(평가된 12개 하니스-조건 조합에서 무작위로 뽑은 경쟁 구성에 대한 과제별 승률 평균)를 함께 보고한다.

세 하니스-모델 구성

- **Codex CLI + GPT-5.5 (high)** — 고추론 설정
- **OpenCode + DeepSeek-V4-Pro**
- **Pi Coding Agent + MiniMax-M3**

비교 대상과 프로토콜

기준선은 **Vanilla**, 즉 같은 하니스-모델 구성을 HoH 프로토콜 없이 한 번 실행한 것이다. Vanilla는 표준 개발 패스 한 번이고, $\text{HoH}@T$ 는 계획-코딩-테스트 반복을 T 회 돌리며 산출물과 실행 증거를 반복 사이에 운반한다. 본 실험은 $T = 3$ 이다. 프로토콜의 공정성 조건은 두 가지다. 첫째, Vanilla와 HoH가 같은 벤치마크 제공 초기 상태와 같은 하니스-모델 구성을 쓰고 HoH 프로토콜 적용 여부만 다르다. 둘째, 중간·최종 산출물 평가는 **실행 전체 종료 후에만** 이뤄지고 평가자 출력은 개발 루프로 돌아가지 않는다. 평가 신호의 누수를 막는 장치다(제2장의 증거번들 제한과 짝을 이룬다).

지표

벤치마크	지표	정의
GameCraft-Bench	Overall (0-100)	컴파일·실행 실패는 0점, 실행 가능 산출물은 Core Mechanics·Content Depth·Functional Visuals·Art and Presentation을 벤치마크 가중치로 합산
FrontierSWE	공식 보상 + Dominance	과제별 검증자가 매기는 공식 보상의 평균, 그리고 경쟁 구성 대비 승률
ProgramBench	Avg. Test Pass Rate	과제별 은닉 동작 테스트 통과율의 과제 평균

부가 신호로 코딩 하니스 모델 호출의 누적 입출력 토큰을 제공자 보고치로 기록한다(벤치마크 평가 호출 제외). 입력 토큰에는 캐시 문맥 읽기가 섞일 수 있어 제공자 간 비용 직접 비교가 아닌 **구성 내 비교**에만 쓴다.

이 설계의 관점 하나를 짚고 넘어가자. 논문이 묻는 것은 “HoH 프로토콜이 같은 하니스에 얼마를 얻는가”다. 모델을 바꾸거나 하니스를 바꾼 효과가 아니다. 반복 구조 자체의 기여를 분리해 내는 통제 설계다.

DAY

05

제5장 — 숫자가 말하는 것

요약표 — 세 벤치마크, 세 구성, 반복별

논문 Table 1에서 대표 지표만 뽑아 옮긴다. GameCraft-Bench는 Overall(0~100), FrontierSWE는 Dominance(%), ProgramBench는 Avg. Test Pass Rate다.

구성	설정	GameCraft Overall	FrontierSWE Dominance	ProgramBench Pass Rate
Codex + GPT-5.5 (high)	Vanilla	49.58	46%	60.41
	HoH@1	59.71	45%	65.42
	HoH@2	64.84	49%	65.79
	HoH@3	71.52	67%	66.50
OpenCode + DeepSeek-V4-Pro	Vanilla	26.90	26%	45.27
	HoH@1	28.61	34%	55.33
	HoH@2	40.32	47%	55.66
	HoH@3	48.98	49%	57.56
Pi + MiniMax-M3	Vanilla	42.16	36%	35.83
	HoH@1	49.06	65%	48.68
	HoH@2	55.04	68%	53.57
	HoH@3	58.78	66%	52.68

HoH@3는 세 벤치마크 전부, 세 구성 전부에서 Vanilla를 넘는다. GameCraft-Bench Overall 향상은 Codex +21.94, OpenCode +22.08, Pi +16.62점이다. ProgramBench Pass Rate는 +6.09~+16.85점, FrontierSWE 보상은 +0.07~+0.29다.

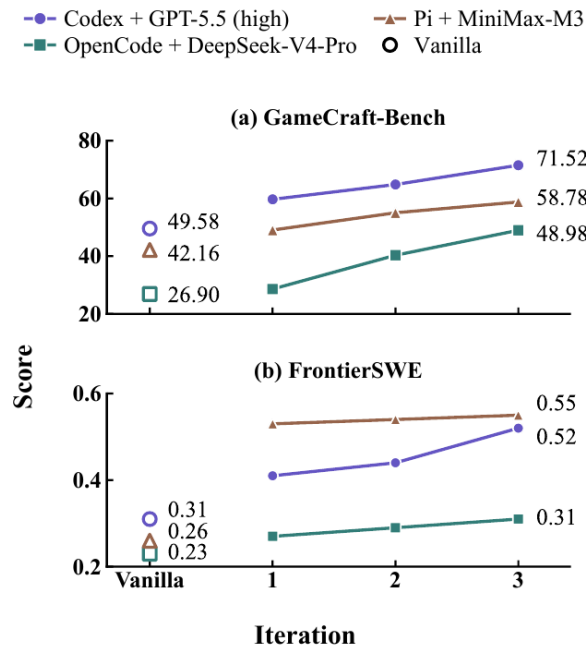


Figure 3 | Overall performance across three iterations on GameCraft-Bench and FrontierSWE, with Vanilla shown for reference.

그림 5-1 반복별 점수 — 세 구성 모두 반복 3까지 꾸준히 오른다

반복은 계속 이득인가

그림 5-1처럼 세 구성의 GameCraft 점수는 반복 1→3 내내 오른다. Pi 구성의 FrontierSWE Dominance는 반복 1에서 이미 36%→65%로 뛰고 2에서 68%로 정점을 찍는다. 반복이 늘수록 이득이 꺾이는 포화 흔적이 일부 구성에 보이지만 (GameCraft에서 HoH@2→3 상승폭이 HoH@1→2보다 작다), 어느 구성에서도 반복을 늘려서 손해 보는 셀은 없다.

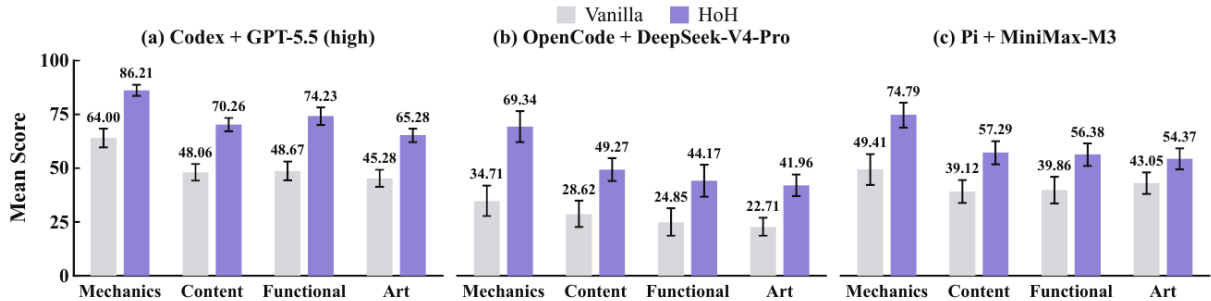


Figure 4 | GameCraft-Bench component scores for Vanilla and HoH ($T = 3$). Results are reported mean score for Core Mechanics, Content Depth, Functional Visuals, and Art and Presentation with 95% bootstrap confidence intervals.

그림 5-2 GameCraft-Bench 컴포넌트 점수 — 네 컴포넌트 모두 HoH가 앞선다

컴포넌트를 뜯어도 같은 방향이다(그림 5-2). Core Mechanics, Content Depth, Functional Visuals, Art and Presentation 네 항목 모두에서 HoH@ $T = 3$ 이 Vanilla를 앞선다. 특히 Codex 구성에서 Mechanics는 48.67→86.21로 가장 크게 벌어진다. 반복 개선이 걸모습만 다듬는 게 아니라 게임의 뼈대(메커니즘)까지 끌어올린다는 뜻이다.

정성 비교 — 무엇이 실제로 달라졌나

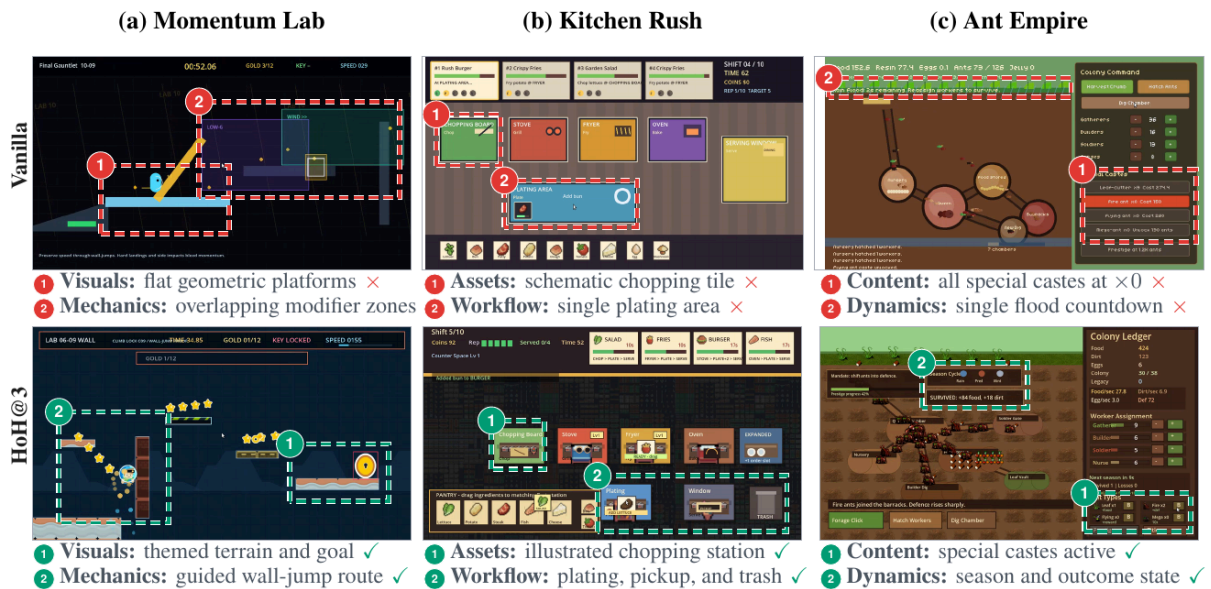


Figure 5 | Qualitative comparison of final game artifacts produced by Vanilla and HoH@3 using Codex with GPT-5.5 (high). Columns show three GameCraft-Bench tasks from distinct game families: Momentum Lab (momentum-based platformer), Kitchen Rush (restaurant-management simulation), and Ant Empire (idle colony-management game). Rows show gameplay frames from Vanilla (top) and HoH@3 (bottom). Numbered dashed boxes identify the regions discussed in the annotations; red crosses and green checks denote limitations and implemented functionality, respectively.

그림 5-3 Vanilla 대 HoH@3 최종 게임 산출물 비교 — 빨간 X는 한계, 초록 체크는 구현된 기능

그림 5-3은 Codex + GPT-5.5 구성의 최종 산출물 세 게임을 나란히 놓는다.

- **Momentum Lab**(모멘텀 플랫폼): Vanilla는 평면적 기하 플랫폼에 수정 영역이 겹치는 결함. HoH@3는 테마 지형과 목표 표시, 유도된 월점프 경로를 구현.
- **Kitchen Rush**(레스토랑 경영 시뮬레이션): Vanilla는 개략적인 도마 타일에 조리 구역이 하나뿐. HoH@3는 삽화 도마 스테이션에 조리·픽업·쓰레기 처리 워크플로가 통째로 들어옴.
- **Ant Empire**(유류 군체 경영): Vanilla는 특수 계급이 전부 $\times 0$ 배율로 사실상 부재. HoH@3는 특수 계급이 실제 활성화되고 계절과 결과 상태가 동작.

공통 패턴이 보인다. Vanilla 산출물은 **표면은 있고 내부가 비어 있다**. 존재하지만 배율 0인 기능, 그려졌지만 이어지지 않는 워크플로. HoH 반복은 이 빈 곳을 채운다. 이것이 증거번들의 gap 레코드가 다음 반복의 수정 대상으로 작동한 결과다.

예산 통제 비교와 장기 지속성

같은 반복 횟수를 단순 반복 실행(Vanilla continuation)으로 돌리는 **패스 통제 비교**에서도 HoH가 앞선다. 같은 토큰 예산을 줘도 마찬가지다. 반복 구조가 이득인 지점은 추가 호출량이 아니라 **호출 사이클**을 잇는 **상태임**을 보여주는 대조다.

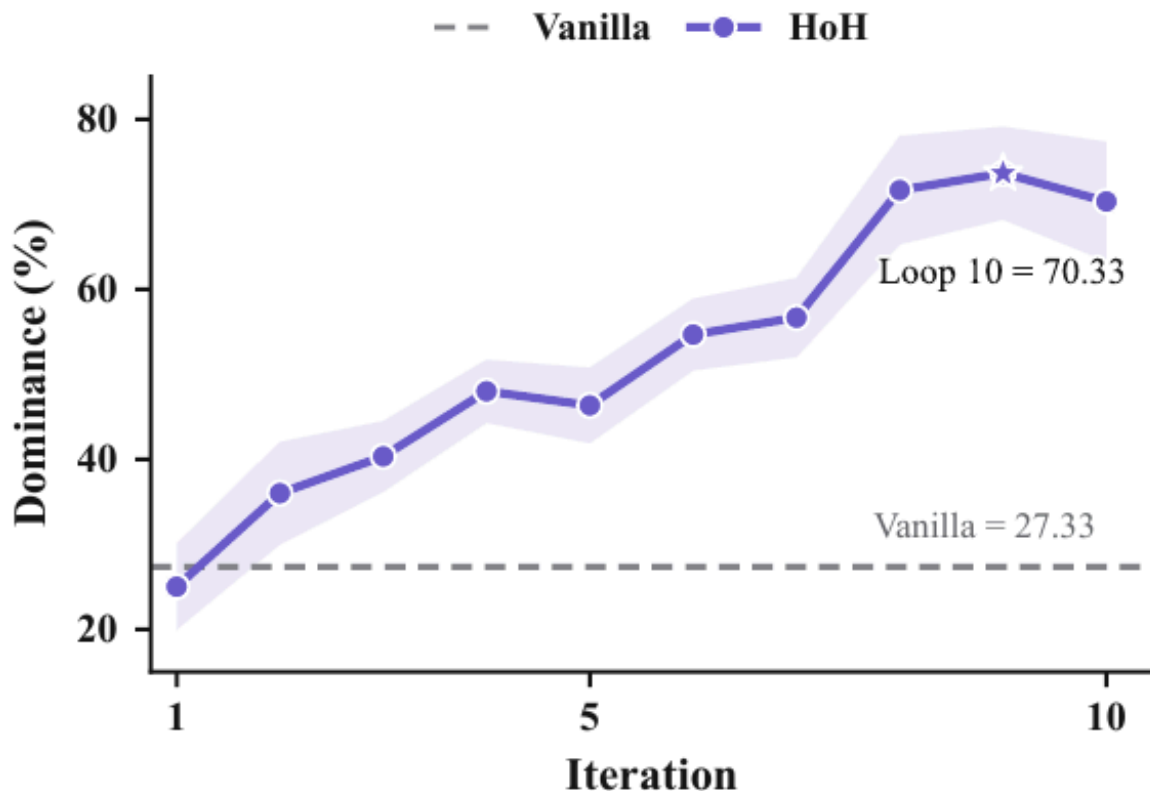


Figure 6 | HoH gains persist at iteration 10. Across 15 FrontierSWE tasks, the band shows ± 1 SE; the star marks the best checkpoint and the dashed line Vanilla.

그림 5-4 반복 10까지 이어진 Dominance — HoH 70.33% 대 Vanilla 27.33%

FrontierSWE 15과제에서 반복을 10까지 늘린 추가 실험(그림 5-4)은 이 그림을 확정 짓는다. HoH Dominance는 약 25%에서 시작해 반복 10에서 70.33%까지 오르고, Vanilla 기준선은 27.33%에 머문다. 이득은 반복 3에서 멈추지 않는다.

정리하면 이렇다. HoH의 향상은 한 구성의 우연이 아니라 (1) 세 벤치마크, (2) 세 하니스-모델 구성, (3) 네 품질 컴포넌트, (4) 반복 10까지의 장기 구간에서 일관되게 나타난다. 바뀐 것은 모델도 하니스도 아니고, 반복 사이에 무엇을 남기는 가였다.

DAY

06

제6장 — 진행 중인 사례: 퓨즈포인트

벤치마크 점수 다음으로, 논문은 1인칭 슈팅 게임 **Fusepoint**의 다일 자율 개발을 중단 사례로 진행하고 있다. 이 장을 읽을 때 한 가지를 먼저 짚는다. **이 사례 연구는 원문 집필 시점에도 진행 중이다**. 감사 표와 플레이테스트 점수는 TBD 자리표시자이고, 이슈 곡선의 뒷구간은 투영값이다. 이 책도 확정되지 않은 수치를 확정된 것처럼 쓰지 않는다.

■ 설정 — 사람이 남긴 것과 남기지 않은 것

사용자가 PRD를 넘겨준 뒤로는 계획, 구현, 디버깅, 검증에 사람의 개입이 전혀 들어가지 않았다. 사람이 개입한 것은 할당량·외부 서비스 가용성 복구뿐이며, 이 개입은 산출물을 바꾸지도 개발 계획을 꺾지도 않았다. 논문이 주장하는 자율성의 범위는 개발 **과정**이지, 지원 인프라의 무중단 운영이 아니라는 점이 정확하다.

에셋 생산은 별도의 병목으로 남았다. 3D 에셋 생성 시도는 비용이 크고 게임에 요구되는 시각 품질에 안정적으로 닿지 못했다. 최종 산출물은 CC0·CC BY 등 개방 라이선스 에셋을 쓰고, 출처·라이선스 메타데이터·요구되는 귀속 표기를 보존한다. 이 사례의 교훈은 담백하다. 생산 품질 에셋을 전부 중단 생성하는 것보다 라이선스 콘텐츠를 통합하는 쪽이 신뢰할 만했다.

■ 이슈 장부 — 발견과 재발의 궤적

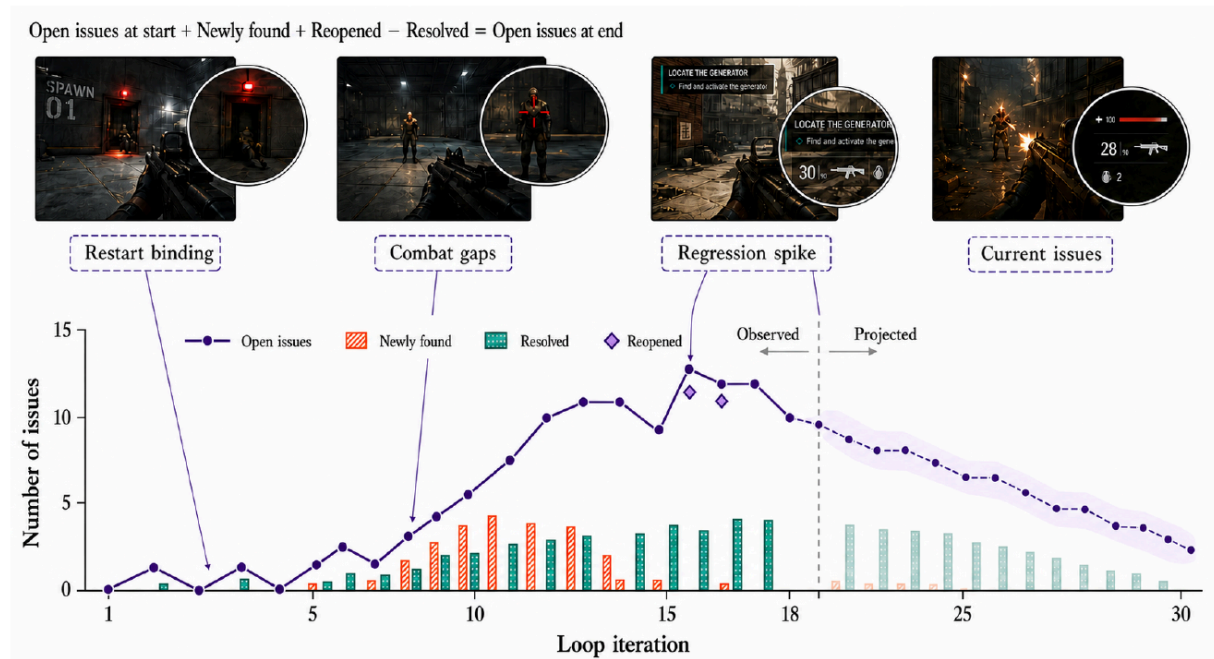


Figure 7 | Issue evolution during the ongoing Fusepoint case study. The solid line reports open issues at the start of Loops 1–18; bars show newly found and resolved issues, and diamonds mark reopened issues. The dashed segment and faded bars are projected placeholders through Loop 30. The FPS callouts are synthetic layout placeholders pending replacement with execution screenshots.

그림 6-1 Fusepoint 이슈 진화 — 루프 1-18 관측, 19-30은 투영(점선). 마름모는 재개된 이슈

이슈 장부를 루프 단위로 추적하면 개발 루프가 구현 문제를 발견하고 해결하고 때로는 **다시 열**는 궤적이 요약된다(그림 6-1). 실선은 루프 1-18 시작 시점의 미해결 이슈 수, 막대는 신규 발견·해결 수, 마름모는 재개된 이슈다. 점선 구간과 열은 막대는 루프 30까지의 투영 자리표시자다. 최종 감사가 끝나면 실행 스크린샷으로 교체될 예정이다.

■ 최종 평가 — 세 층으로 나눈 증거

진행 중인 평가 설계 자체가 배울 점이다. 논문은 최종 산출물의 증거를 세 층으로 분리한다.

1. **납품 가능성 게이트** — 깨끗한 빌드, 성공적 실행, 중단 완주, 실패/재시작/리플레이, 반복 실행 안정성.

2. **명시된 PRD 요구 이행** — 월드·에셋 통합, 미션 흐름, 전투·적 AI, 조작·UI·피드백, 연출·완성도.

3. **PRD 너머의 기여** — 추가 기능, 선제적 버그 수정, 경험 개선. 별도 집계.

이 분리의 의도는 명확하다. **선택적 확장이 미충족 요구를 가려서는 안 된다**. 예쁜 부가 기능 목록이 핵심 요구 누락을 덮는 식의 자기보고를 구조적으로 막는다. 감사 표는 HoH의 자기평가(보고됨)와 독립 감사 결과(검증됨)를 나란히 두고, 뒷받침 없는 완료 주장에 경고 표시를 붙인다.

플레이 경험 평가도 준비돼 있다. 출처를 숨긴 소규모 플레이테스트(3~5명, FPS 경험자와 CS·SW·게임개발 배경 혼합)에서 평가자는 같은 빌드와 과제 스크립트로 메인 미션을 한 번 클리어하고 자유 플레이 뒤 채점하며, **AI 개발 출처는 채점 제출까지 공개되지 않는다**. 채점 도구는 PXI(Player Experience Inventory) 30문항 코어 10구성개념 + 별도 Enjoyment 3문항이다. 표본이 작아 기술통계와 개인별 평점만 보고하고 다차원 도구를 단일 점수로 뭉치지 않는다.

■ 지금 시점에서 배울 수 있는 것

결과가 TBD여도 설계는 확정돼 있다. 이 사례가 남기는 것은 세 가지다. 첫째, 다일(數日) 자율 개발이 실제로 수십 루프 규모로 운영됐다는 존재 증명. 둘째, 사람 개입을 **인프라 복구로 한정**하는 자율성 정의 — 산출물 개입이 아니라. 셋째, 자기평가와 독립 감사를 갈라놓는 평가 구조. 마지막 하나는 벤치마크 점수를 넘어서는 실전 자율 에이전트의 신뢰 문제에 대한, 가장 실용적인 답이다.

DAY

07

맺음말 — 우리 시스템에 남는 것

논문을 한 문장으로 압축하면 이렇다. **하니스가 모델을 구조화하듯, HoH는 하니스의 참여를 구조화한다.** 기존 하니스를 고치지 않고, 반복 구조와 두 상태 채널(산출물 계보, 증거번들)만 얹어 세 벤치마크·세 구성에서 일관된 향상을 냈다. 이 결론을 우리 시스템에 비추면 남는 것이 몇 개 있다.

하나 — 파이프라인 스킴에는 바깥 루프가 필요하다

우리 korean-ebook 파이프라인은 `build.py` → `qc_gate.py`라는 결정론 고리를 이미 갖고 있다. 이것의 위치를 다시 보자. 빌드·게이트 스크립트는 HoH의 코딩·테스트 단계에 해당하고, 에이전트(우리)는 계획 단계에 해당한다. 게이트가 FAIL을 내면 `gate-report.json`이 gap 레코드고, 지적된 면만 수정 후 재빌드하는 우리의 관행이 곧 `Plan_H(S, E_{t-1})`였다. 우리는 프레임워크의 형태를 이미 쓰고 있었다. 다만 그것을 상태로 명문화한 적은 없었다.

그 빈자리를 SKILL.state 적용(v0.2.0)으로 채웠다. 체크 판정마다 `work/state.json`에 상태 패치를 남기고, 재개는 대화 히스토리 유추가 아니라 next 필드부터 시작한다. 이 책의 언어로 말하면, 컨텍스트는 호출 안에서만 살고 **기록은 반복 사이를 산다는** 교훈 5의 구현이다.

둘 — 증거는 주장에 붙여 다녀야 한다

HoH의 증거번들은 자유 서술 보고서가 아니라 (주장, 관측 기록, 판정)의 쌍이다. 우리 시스템에 대응시키면, `qc_gate.py`의 `gate-report.json`이 G1-G5 각 게이트별 판정과 근거를 구조화해 내놓는 방식과 같은 뼈대다. “고쳤다”는 서술보다 “어느 주장이 어느 관측으로 verified/gap인가”가 다음 계획의 입력이 된다. 리뷰·검증 서브에이전트의 산출물 설계에도 같은 원리를 쓸 수 있다. 발견마다 관측 근거와 판정을 붙여 내놓는 구조 말이다.

셋 — 역할은 결정 경계로 나눈다

계획자는 쓰지 않고, 개발자는 쓰고, QA는 읽고 실행만 한다. 이 권한 계층이 완료 판정을 자가 점검에 두지 않게 한다. 우리의 `klic-effort-router`가 계획·검토·구현·리뷰·검증을 화이트리스트 서브에이전트로 갈라놓는 관행과 같은 방향이다. 역할을 추가할 때 물을 질문도 그대로 쓴다. 이 역할이 **독립된 권한과 별도 검증 가능한 판단**을 내놓는가? 아니면 추가하지 않는다.

넷 — 납품 계약은 엄격하게, 실행은 유연하게

HoH는 호출 경계에서 인계 형식과 공유 상태 진입 조건을 정하고, 호출 안의 도구 사용과 구현 경로는 하니스 손에 남긴다. 우리의 qc 게이트가 “PASS일 때만 final/을 만든다”는 것과 “빌드 로그 판단 사유는 스크립트가 출력한다”는 것의 관계다. 무엇을 납품하는지는 엄격하게, 어떻게 만드는지는 유연하게. 이 균형이 깨지는 순간 — 납품 제약이 내부 실행까지 규정하기 시작하면 — 시스템은 깨지기 쉬워진다.

다섯 — 평가 신호는 개발 루프에 흘려보내지 않는다

HoH는 평가를 실행 종료 후에만 하고 평가자 출력을 루프로 돌리지 않는다. 과적합을 막는 장치다. 우리 시스템에서 이것은 기준선을 정해두고 작업물 스스로를 기준선에 맞춰 튜닝하는 일을 경계하라는 뜻이 된다. 퓨즈포인트 사례의 자기평가·독립 감사 분리도 같은 문제에 대한 답이다.

—

그린필드 개발에서 하니스가 버티는 시간은 모델 능력의 문제이기 전에 상태 설계의 문제였다. 반복 사이에 산출물과 증거를 남기는 하니스는 며칠을 간다. 아무것도 남기지 않는 하니스는 한 번의 실행으로 끝난다. 이 책이 우리 시스템에 남기는 것도 결국 한 줄이다. **다음 반복이 상속받을 것을, 지금 상태에 적어 두어라.**